



— NAVAIAFORGE · TECHNICAL TRACK

Build, Run & Publish an AI Workforce

Run the NavaiaForge backend on your machine, assemble a multi-agent workforce with the SDK, prove it works, then sync it to the cloud and publish it to the marketplace.

Assessment — AI Engineer Intern

ROLE	AI Engineer Intern
TRACK	NavaiaForge SDK (Forge)
FORMAT	Hands-on, take-home
YOU WILL DELIVER	A workforce live in the marketplace
SUPPORT	info@navaia.sa

n a v a i a . s a

ABOUT NAVAIA

An AI workforce for Saudi businesses

Navaia builds AI 'workforces' — teams of AI agents that do real operational work for Saudi companies. The voice agent runs calls, the chat agent handles WhatsApp, the HR agent screens candidates, and Forge lets engineers assemble and sell multi-agent workforces of their own.

The products you can reference

Voice Agent — Bilal

agentic.navaia.sa

AI voice assistant for high-volume inbound & outbound calls: collections, bookings, tickets, campaigns, CRM, analytics, and AI call-quality analysis. Speaks Saudi & Gulf dialects.

Chat Agent — Baian

baian.navaia.sa

WhatsApp Business agent: answers customer chats 24/7 from a knowledge base, routes to departments (sales, support, HR), flags messages that need a human, and tracks customers, users, and analytics.

HR Agent — Niqwa

niqwa.navaia.sa

Screens candidates, schedules interviews, answers repetitive applicant questions, and organises evaluation notes for fast-hiring teams.

Forge marketplace

fareegi.navaia.sa

Build multi-agent 'workforces' with the Forge SDK, run them, and publish them to a marketplace where others can install — and you can earn.

Explore the products (use these to make your work real)

Website: navaia.sa — company, products, positioning.

Forge: fareegi.navaia.sa — the workforce marketplace.

Where this assessment lives. You work in **Forge** — `fareegi.navaia.sa` is the cloud + marketplace. The SDK runs a backend locally for execution, then syncs results to the cloud so your workforce can be published. This task has you do exactly what a real Forge builder does, end to end.



The Assessment

Backend → SDK → workforce → run → sync → publish.

Best for: AI Engineer Intern

Hands-on take-home

SDK, agents, Docker, marketplace

Purpose. Build a workforce, run it, and publish it to the marketplace — the same loop a Forge builder runs every week. We want to see that you can read a real SDK, get a backend running, design a sensible multi-agent workforce, and ship it.

How to approach this — don't overthink it. The phases below are an **example of the shape** of the work, not a rigid script you must follow step by step. The easy path: take the markdown version of this brief (provided alongside this PDF) and hand it to your coding agent — Navaia Code or Claude Code — and it will walk you through every detail. All you need to bring is the **idea for the workforce** you want to build. Keep it simple.

What you'll do

1. Run the NavaiaForge backend locally (Docker)
2. Install the SDK and get your API keys
3. Create a workforce with 2–10 agents
4. Run tasks and chat with your agents
5. Sync to the cloud and publish — then see it appear in the marketplace

Prerequisites

- Docker 24+ with Compose v2
- Python ≥ 3.10
- An OpenRouter API key with credits loaded (openrouter.ai/keys)

No license token is required — just bring your OpenRouter key.

Phase 1 — Run the backend

Start the backend container. All agent execution, LLM calls, and data storage happen here.

```
# The compose file ships with the SDK repo (docker-compose.dist.yml).
# If you didn't clone it, download the file:
curl -fLO https://raw.githubusercontent.com/NavaiaSolutions/navaia-forge-sdk/main/docker-
compose.dist.yml

# Create your .env (or copy from .env.example)
cat > .env <<'EOF'
SECRET_KEY=$(openssl rand -hex 32)
POSTGRES_USER=navaia_forge
POSTGRES_PASSWORD=change-me-please
POSTGRES_DB=navaia_forge
API_PORT=8001
OPENROUTER_API_KEY=sk-or-v1-your-key-here
WEAVIATE_API_KEY=$(openssl rand -hex 32)
ALLOWED_ORIGINS=http://localhost:3030,http://localhost:3000
EOF

docker compose -f docker-compose.dist.yml up -d
curl http://localhost:8001/health
```

Local backend **http://localhost:8001**. The backend uses your `OPENROUTER_API_KEY` for both chat and task execution — you pay OpenRouter directly; NavaiaForge never touches your model credentials.

Windows users: the `.env` block uses bash syntax. On PowerShell or CMD, create the `.env` by hand with the same key=value pairs, and generate `SECRET_KEY` with `python -c "import secrets; print(secrets.token_hex(32))"`.

Phase 2 — Install the SDK & get your API key

```
pip install navaia-forge
```

Create an account and a long-lived API key from the SDK:

```
from navaia_forge import NavaiaForgeClient
client = NavaiaForgeClient(base_url="http://localhost:8001")

# Register the first time (use client.auth.login(...) after that)
pair = client.auth.register(name="Your Name", email="you@example.com", password="your-password")

authed = NavaiaForgeClient(base_url="http://localhost:8001", api_key=pair.access_token)
key = authed.auth.create_key("my-key")
print(key.api_key) # nf_... — stored once, copy it now
```

Phase 3 — Build your workforce (2–10 agents)

The workforce should solve a real problem. Design your own (sales, research, support, DevOps, content...), adapt a prior agent project, or recreate a compelling multi-agent setup you find online. It must be reasonable and functional — it should produce useful output when given a task.

```
from navaia_forge import NavaiaForgeClient
local = NavaiaForgeClient(base_url="http://localhost:8001", api_key="nf_...")

wf = local.workforces.create(name="Research Team", runtime_mode="navaia_code")

researcher = local.agents.create(workforce_id=wf.id, name="Researcher", role="research",
    instructions="Find and summarize information on the topic. Provide sources and key findings.",
    model_provider="openrouter", model_name="anthropic/claude-sonnet-4")

writer = local.agents.create(workforce_id=wf.id, name="Writer", role="writer",
    instructions="Turn the findings into a clear brief under 500 words.",
    model_provider="openrouter", model_name="anthropic/claude-sonnet-4")

local.workforces.edges.create(workforce_id=wf.id,
    source_agent_id=researcher.id, target_agent_id=writer.id)
```

Use the full OpenRouter model ID (e.g. `anthropic/claude-sonnet-4`, not just `"sonnet"`); browse models at openrouter.ai/models. You are not limited to this shape — use as many agents and edges as your use case needs, within 2–10 agents.

Phase 4 — Run and verify

Run a task:

```
task = local.tasks.create(workforce_id=wf.id,
    title="Survey recent advances in multi-agent AI systems")
final = local.tasks.wait_for_completion(task.id)
print(final.status, final.result)
```

Or chat with your agents through a conversation:

```
convo = local.conversations.create(workforce_id=wf.id)
local.conversations.send_message(convo.id,
    "What are the top 3 trends in multi-agent AI right now?")

import time
time.sleep(10)
for m in local.conversations.messages(convo.id):
    print(f"[{m.role}]: {m.content}")
```

The printed status and result, and the returned messages, confirm your workforce works.

Phase 5 — Sync to cloud & publish

Create a cloud account at `fareegi.navaia.sa` (**Get Started**), generate a key under **Settings > Manage API keys**, then sync and publish:

```
cloud = NavaiaForgeClient(base_url="https://fareegi.navaia.sa", api_key="nf_cloud_...")

result = local.sync.push("your-workforce-id", remote=cloud)
print(f"Synced: {result.action}, cloud ID: {result.workforce_id}")

cloud.workforces.publish(result.workforce_id,
    tagline="A concise description of what your workforce does",
    category="research",
    price_cents=0,          # 0 = free, or set your price in cents
    currency="SAR")
```

Pick an appropriate category — the marketplace uses Research, Content, Sales, Support, Engineering, Business Ops, and QA. Once published, your workforce enters a short moderation queue; after the Navaia team verifies it, **it appears as a card in the marketplace** at `fareegi.navaia.sa`, alongside the other published workforces — that visible listing is the finish line. Free workforces (`price_cents=0`) are installable immediately.

CLIENT	BASE URL	PURPOSE
local	<code>http://localhost:8001</code>	Your backend — all execution and data live here.
cloud	<code>https://fareegi.navaia.sa</code>	Display layer — marketplace, dashboard, sharing.



What to submit & how we score it

Done = your workforce is live in the marketplace.

- ☐ A running local backend (/health OK).
- ☐ A workforce of 2–10 agents created via the SDK.
- ☐ At least one completed task or conversation showing the agents work.
- ☐ The workforce synced to `fareegi.navaia.sa`.
- ☐ The workforce **appears in the marketplace** — find your card under the Marketplace (المتجر).
- ☐ A few lines on what your workforce does and why you built it that way — and, if you used AI tools, how.

Evaluation

CRITERION	WEIGHT	WHAT WE LOOK FOR
Gets it running	15%	Backend, SDK, and keys set up correctly and independently.
Workforce design	25%	Agents, roles, and edges make sense for a real problem; not a toy.
Functionality	25%	The task or conversation completes and produces genuinely useful output.
SDK & code quality	15%	Clean, correct use of the SDK; sensible models and instructions.
Shipped	10%	Synced and published with a clear tagline, category, and price.
Communication	10%	The short write-up explains decisions clearly and honestly.

Tip. A focused 2–4 agent workforce that clearly works and is well-explained beats an ambitious 10-agent one that half-runs. Pick a problem you understand, make it produce something real, then ship it.

Before you start. You may use AI tools, but the final judgement, structure, and editing must be your own — if you use AI, add one line on how you used it. This assessment is for evaluation only: keep it original, do not include confidential information from any company or client, and note that Navaia will not publish your work without your permission.